AI Security Findings

Overall risk: CRITICAL | Total findings: 17

Executive summary

Risk overview

Total findings: 17. Critical: 5, High: 8, Medium: 4, Low: 0.

Technical summary

Overall Risk Level: Critical

Key Issues to Fix First:
1. Command Injection Vulnerabilities – Multiple endpoints (e.g., /run) allow unsanitized user input to be passed directly to system commands. This must be immediately remediated using safe APIs and rigorous input validation.
2. SQL Injection Vulnerabilities – Several functions and endpoints concatenate user input into SQL queries without proper sanitization or parameterization. Refactor these areas to use prepared statements/parameterized queries.
3. Hardcoded Secrets and Credentials – Critical secrets such as API keys, database credentials, JWT secrets, and tokens are embedded directly in the source code and configuration files. Transition these to secure storage solutions (environment variables or a dedicated secrets manager).
4. Insecure Token Verification – The current token verification logic (accepting tokens based solely on a prefix match) and the usage of hardcoded tokens pose significant authentication risks. Revise the authentication mechanism to incorporate robust cryptographic validation.

Quick Wins & Hardening Recommendations:
• Disable DEBUG mode in production to prevent leakage of sensitive internals.
• Remove log entries that capture raw passwords or other sensitive data; ensure that sensitive details are masked in logs.
• Sanitize web inputs to mitigate Cross-Site Scripting (XSS) vulnerabilities especially in client-side dynamic content.
• Consider a comprehensive review of existing security controls to adopt best practices (e.g., least privilege access, proper error handling, and input validation) across the application.

Immediate remediation of these issues, particularly those enabling command and SQL injection, should be prioritized to reduce the immediate threat surface.

AI narrative (LangChain)

Below is a summary report of the security review:

Overall Risk Level: Critical

Multiple high-risk vulnerabilities were discovered during the full security scan. Immediate remediation is needed to reduce the risk level. The following are the most critical findings:

1. Command Injection
• The /run endpoint and related functions pass unsanitized user input directly to os.popen(), allowing an attacker to execute arbitrary commands on the server.
• Recommendation: Use secure alternatives such as the subprocess module with validated inputs, avoiding direct shell access.

2. SQL Injection
• Several endpoints (like /user, get_user, and get_user_raw_query) build SQL queries using unsanitized user inputs, exposing the system to SQL injection attacks.
• Recommendation: Transition to parameterized queries or employ an ORM to properly escape inputs.

3. Hardcoded Secrets & Credentials
• The code contains hardcoded API keys, JWT secrets, database credentials, and other sensitive data.
• Recommendation: Remove hardcoded secrets immediately. Utilize secure storage like encrypted environment variables or dedicated secrets management solutions.

4. Insecure Token Verification
• The authentication mechanism uses hardcoded tokens and insufficient validation, making the system vulnerable to bypass attacks.
• Recommendation: Implement a robust authentication framework using standard techniques (e.g., JWT with a strong signing algorithm) and ensure tokens are properly validated.

5. Information Exposure & Debug Configurations
• The application is running in debug mode and logs plaintext sensitive information, potentially exposing critical details.
• Recommendation: Disable debug mode in production, avoid logging plaintext passwords, and secure endpoints that potentially leak configuration details.

Next Steps for Remediation:

• Secure Input Handling:
– Audit all endpoints and validate/sanitize user inputs.
– Replace unsafe operations related to command execution and SQL query construction with secure alternatives.

• Reinforce Authentication:
– Replace weak token handling with industry-standard libraries (e.g., properly implemented JWT).
– Ensure that tokens are dynamically created and securely validated.

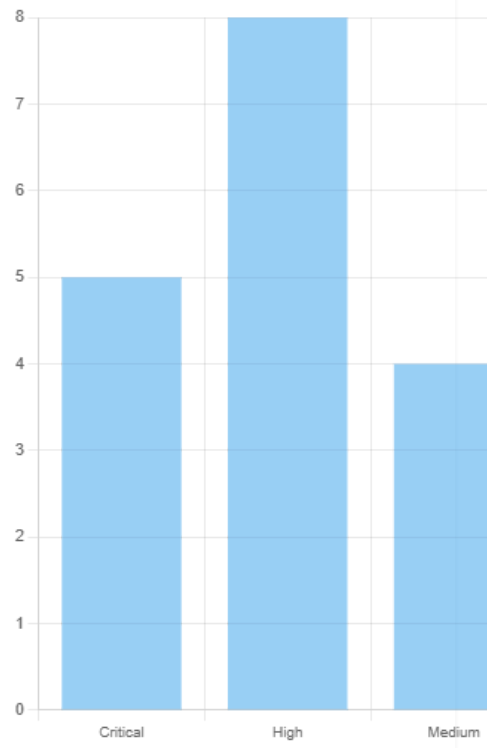
• Eliminate Hardcoded Secrets:
– Identify and remove all hardcoded credentials from the source code.
– Migrate secrets to secure storage solutions.

• Revise Debug and Logging Settings:
– Immediately disable debug mode in production.
– Adjust logging configurations to prevent the exposure of sensitive data.

• Follow-up Security Testing:
– After remediation, perform a secondary security audit and consider professional penetration testing.

Prioritizing fixes for the command injection and SQL injection issues is crucial, as these are high-risk problems that could lead to complete system compromise. Addressing these recommendations will significantly improve the application's security posture.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
MEDIUM	Debug Mode Enabled	`app.config["DEBUG"]`	Running the application in DEBUG mode in production may reveal sensitive application internals and stack traces to an attacker, increasing the attack surface.	Disable debug mode in production by setting DEBUG to False.
HIGH	Hard-coded Secret	`API_SECRET variable`	The API secret key is hard-coded in the source code, which could be exposed if the code is leaked or accessed by unauthorized users.	Store secrets in environment variables or secure secret management systems and load them during runtime.
CRITICAL	Command Injection Vulnerability	`app.route('/run')`	User input from the 'cmd' parameter is directly passed to os.popen without validation, allowing an attacker to execute	Avoid executing system commands with unsanitized user input. Use safe APIs or properly validate and sanitize any inputs if command execution is absolutely necessary.

Severity	Title	File / Location	Description	Recommendation
			arbitrary system commands.	
CRITICAL	SQL Injection Vulnerability	`app.route('/user')`	User input from the 'id' parameter is passed to get_user_raw_query without proper sanitization, potentially allowing SQL injection if that function builds queries using concatenation.	Use parameterized queries or prepared statements to safely handle user input in database queries.
MEDIUM	Insecure Token Verification	`app.route('/login')`	The token verification function (verify_token_insecure) is flagged as insecure. Depending on its implementation, it may allow unauthorized access.	Implement a secure token verification mechanism using robust libraries, and ensure that tokens are signed and validated properly.
MEDIUM	Sensitive Configuration Exposure	`app.route('/config')`	The endpoint exposes sensitive configuration information by reading and returning the contents of the 'config/settings.json' file, which may contain secrets or sensitive data.	Restrict access to configuration files, avoid exposing them via public endpoints, or sanitize sensitive information before returning any configuration data.
HIGH	Hardcoded Tokens (Insecure Credential Storage)	`HARDCODED_TOKENS`	The code uses a hardcoded list of tokens for authentication. Hardcoding credentials increases the risk of exposure, especially if the source code becomes available to unauthorized parties. Additionally, tokens like 'admin-token' and '12345' are too predictable and can be easily guessed.	Remove hardcoded tokens and use a secure credential storage mechanism. Consider leveraging environment variables, secure configuration management, or a dedicated authentication service that can manage token lifecycle and revocation.
CRITICAL	Insecure Token Verification Logic	`verify_token_insecure`	The token verification logic accepts any token that starts with 'test-'. This overly permissive check can allow unauthorized access if an attacker submits any token with the 'test-' prefix, bypassing intended security controls. The logic also fails to incorporate cryptographic checks that are standard for token verification.	Revise the authentication logic to reject tokens based solely on a prefix match. Implement a secure token verification process that includes cryptographic validation (e.g., using HMACs or JWT signature verification) and ensure that tokens are generated and validated in a secure manner.
HIGH	Hardcoded Database Credentials	`Configuration File (line 1-3)`	The configuration file contains hardcoded credentials (DB_HOST, DB_USER, and DB_PASSWORD) which can lead to unauthorized database access if the file is exposed. Using 'root' as the default user with a	Remove sensitive credentials from source code and store them securely using environment variables or a dedicated secrets management tool. Additionally, use least privilege access for database connections.

Severity	Title	File / Location	Description	Recommendation
			simple password increases risk significantly.	
HIGH	Hardcoded JWT Secret	`Configuration File (line 4)`	The JWT secret is hardcoded in the configuration file, which can compromise the JWT-based authentication. An attacker with access to the secret can create and forge valid tokens, bypassing authentication and authorization controls.	Store the JWT secret in a secure external configuration, such as environment variables or a dedicated secrets manager. Ensure strong rotation policies and robust key management practices.
HIGH	SQL Injection Vulnerability	`get_user function (line where query string is built)`	The function concatenates the 'uid' parameter directly into the SQL query string without any sanitization or parameterization, making it vulnerable to SQL injection attacks. An attacker could pass in a malicious 'uid' to execute arbitrary SQL commands.	Use parameterized queries or prepared statements to safely incorporate user input into SQL queries. For instance, rewrite the query to use placeholders: 'query = "SELECT * FROM users WHERE id = ?"' and then execute it with the tuple containing uid. Also, ensure 'uid' is cast to an integer if appropriate.
HIGH	SQL Injection Vulnerability via Direct String Concatenation	`get_user_raw_query function - query construction`	User input (user_id) is concatenated directly into the SQL query string without proper sanitization. This exposes the application to SQL injection attacks, allowing an attacker to manipulate the SQL query to access or modify data.	Use parameterized queries (prepared statements) to ensure that user input is treated as a data value and not as part of the SQL command. For example, use 'cur.execute("SELECT id, username, email FROM users WHERE id = ?", (user_id,))' in sqlite3.
MEDIUM	Hardcoded API Key	`Global scope variable API_KEY`	The API key is hardcoded in the source code, which can be exposed if the code is leaked or reverse-engineered. This can lead to unauthorized access using the API key.	Remove the hardcoded secret and instead use environment variables or a secure secrets management solution to load sensitive information at runtime.
CRITICAL	Command Injection via os.popen	`Function run() in the '/run' endpoint`	User input from the 'cmd' query parameter is passed directly to os.popen without proper sanitization or validation. This allows an attacker to execute arbitrary operating system commands, potentially compromising the system.	Avoid using unsanitized user input in system commands. Use a safe alternative such as Python's subprocess module with strict parameterization and input validation or a whitelist of safe commands, or remove the functionality entirely if not needed.
CRITICAL	SQL Injection in Query Construction	`Function user() in the '/user' endpoint`	User input from the 'id' query parameter is concatenated directly into an SQL query string, which could allow attackers to inject malicious SQL code if the query is executed against a database.	Use parameterized queries or prepared statements provided by your database library rather than concatenating user input into the SQL query. Validate and sanitize all incoming data, and strictly control the query formulation.

Severity	Title	File / Location	Description	Recommendation
HIGH	Cross-Site Scripting (XSS) via Unsanitized URL Parameter	`document.getElementById('welcome').innerHTML`	The code uses the 'name' parameter from the URL directly in innerHTML without any sanitization or encoding, which can lead to XSS attacks if an attacker passes malicious scripts in the URL.	Avoid inserting untrusted data into innerHTML. Instead, use safe methods such as setting textContent (e.g., document.getElementById('welcome').textContent = 'Hello, ' + name + '!') or properly sanitize/escape the user input before inserting it into the DOM.
HIGH	Sensitive Data Exposure - Logging Raw Password	`log_login_attempt function, logger.warning call`	Passwords (and potentially other sensitive data) are being logged in plain text, which can lead to sensitive data exposure through logs. Logging raw passwords is a security risk as it may allow attackers to retrieve credentials from log files.	Remove logging of sensitive information such as passwords. If logging is necessary for troubleshooting, ensure that sensitive details are masked or avoided entirely. Follow security best practices and guidelines for handling sensitive data.